

Handout

TF-US-OP4

JavaBeans

Inhalt

1	JavaBeans	3
1.1	Was sind JavaBeans?	3
1.2	Anwendungsbeispiel	4
1.3	Erweiterung der Klasse durch weitere Methoden	5
1.4	Überwachung von Änderungen	7
1.5	Client Klasse.....	9

Die Erstellung dieses Handouts erfolgte mit Unterstützung einer KI.

1 JavaBeans

1.1 Was sind JavaBeans?

JavaBeans sind Klassen in Java, die bestimmte Konventionen befolgen. Sie werden hauptsächlich verwendet, um Daten zu kapseln und den Zugriff auf diese Daten zu steuern. Hier sind die Hauptmerkmale eines JavaBeans:

Private Eigenschaften: Die Eigenschaften (Attribute) eines JavaBeans sind privat, was bedeutet, dass sie nicht direkt von außerhalb der Klasse zugegriffen werden können.

Getter und Setter Methoden: Für jede private Eigenschaft gibt es eine öffentliche Getter- und Setter-Methode, um den Wert der Eigenschaft zu lesen bzw. zu setzen.

Standardkonstruktor: Ein JavaBean hat einen Standardkonstruktor ohne Parameter. Dies ermöglicht das Erzeugen eines Objekts ohne die Notwendigkeit, Werte zu initialisieren.

JAVABEANS SIND IN JAVA AUS MEHREREN GRÜNDEN WEIT VERBREITET:

Datenkapselung: JavaBeans bieten eine sichere Methode zur Datenkapselung, indem sie private Datenfelder verwenden und öffentliche Getter- und Setter-Methoden bereitstellen. Dies ermöglicht eine kontrollierte Interaktion mit den Daten und trägt zur Aufrechterhaltung der Datenintegrität bei.

Wiederverwendbarkeit: Durch die Standardisierung ihrer Struktur können JavaBeans leicht in verschiedenen Bereichen einer Anwendung oder sogar über verschiedene Anwendungen hinweg wiederverwendet werden.

Kompatibilität: JavaBeans sind mit einer Vielzahl von Java-Frameworks und -Technologien kompatibel, darunter JSP, Servlets und Spring. Diese Technologien nutzen häufig die Getter- und Setter-Methoden von JavaBeans, um Daten zu manipulieren und zu interagieren.

Serialisierung: JavaBeans unterstützen die Serialisierung, was bedeutet, dass sie in einen Zustand umgewandelt werden können, der leicht über Netzwerke übertragen oder in Dateien gespeichert werden kann. Dies ist besonders nützlich für Techniken wie Remote Method Invocation (RMI) und Sitzungsverwaltung in Webanwendungen.

Einsatz in visuellen Design-Tools: JavaBeans sind so konzipiert, dass sie mit visuellen Design-Tools wie Eclipse's WindowBuilder kompatibel sind. Diese Tools können die Eigenschaften, Ereignisse und Methoden eines JavaBeans inspizieren und manipulieren, um visuelle Komponenten zu erstellen und zu bearbeiten.

1.2 Anwendungsbeispiel

```
public class PersonBean implements Serializable
{
    private String vorname;
    private String nachname;
    private LocalDate geburtsdatum;

    public PersonBean()
    {
    }

    public PersonBean(String vorname, String nachname,
        LocalDate geburtsdatum)
    {
        this.vorname = vorname;
        this.nachname = nachname;
        this.geburtsdatum = geburtsdatum;
    }

    public String getVorname()
    {
        return vorname;
    }

    public void setVorname(String vorname)
    {
        this.vorname = vorname;
    }

    public String getNachname()
    {
        return nachname;
    }

    public void setNachname(String nachname)
    {
        this.nachname = nachname;
    }

    public LocalDate getGeburtsdatum()
    {
        return geburtsdatum;
    }

    public void setGeburtsdatum(LocalDate geburtsdatum)
    {
        this.geburtsdatum = geburtsdatum;
    }
}
```

Abb.1 Beispiel JavaBean Klasse

Der Code implementiert eine Java-Klasse namens `PersonBean`, die das `Serializable`-Interface implementiert. Dies bedeutet, dass Objekte dieser Klasse in einen Byte-Stream umgewandelt und dann wiederhergestellt werden können, was für das Speichern (Persistenz) und das Senden über Netzwerke nützlich ist.

Die `PersonBean`-Klasse hat drei private Attribute:

- `vorname`: Ein `String`, der den Vornamen einer Person darstellt.
- `nachname`: Ein `String`, der den Nachnamen einer Person darstellt.
- `geburtsdatum`: Ein `LocalDate`-Objekt, das das Geburtsdatum einer Person darstellt.

Es gibt zwei Konstruktoren:

- Ein Standardkonstruktor, der keine Parameter akzeptiert.
- Ein Parameterkonstruktor, der `vorname`, `nachname` und `geburtsdatum` als Parameter akzeptiert und diese Werte den entsprechenden Attributen zuweist.

Für jedes Attribut gibt es eine “getter”-Methode (z.B. `getVorname()`) zum Abrufen des Attributwerts und eine “setter”-Methode (z.B. `setVorname(String vorname)`) zum Festlegen des Attributwerts. Diese Methoden sind Teil des JavaBeans-Designmusters, das Kapselung und Datenzugriff in Java fördert.

1.3 Erweiterung der Klasse durch weitere Methoden

```
@Override
public String toString()
{
    return "PersonBean{" +
        "vorname='" + vorname + '\'' +
        ", nachname='" + nachname + '\'' +
        ", geburtsdatum=" + geburtsdatum +
        '}';
}
```

Abb. 2 angepasste Methode `toString()`

Die `toString()` Methode wurde zur `PersonBean` Klasse hinzugefügt. Diese Methode ist eine überschriebene Methode aus der `Object` Klasse in Java und wird verwendet, um eine textuelle Darstellung eines Objekts zu liefern.

In diesem Fall gibt die `toString()` Methode einen `String` zurück, der die Attribute `vorname`, `nachname` und `geburtsdatum` der `PersonBean` in einem bestimmten Format enthält. Dies kann nützlich sein, um Informationen über das `PersonBean`-Objekt auf einfache Weise zu drucken oder zu protokollieren.

```
@Override  
  
public boolean equals(Object o)  
{  
    if (this == o) return true;  
    if (o == null || getClass() != o.getClass()) return false;  
  
    PersonBean that = (PersonBean) o;  
  
    if (!Objects.equals(vorname, that.vorname)) return false;  
    if (!Objects.equals(nachname, that.nachname)) return false;  
    return Objects.equals(geburtsdatum, that.geburtsdatum);  
}
```

Abb.3 angepasste Methode equals

Die `equals(Object o)` Methode wurde zur `PersonBean` Klasse hinzugefügt. Diese Methode ist eine überschriebene Methode aus der `Object` Klasse in Java und wird verwendet, um zu überprüfen, ob zwei Objekte gleich sind.

In diesem Fall überprüft die `equals(Object o)` Methode, ob das aktuelle `PersonBean`-Objekt (`this`) gleich dem übergebenen Objekt (`o`) ist. Hier sind die Schritte, die es durchführt:

1. Überprüft, ob das aktuelle Objekt und das übergebene Objekt identisch sind (`this == o`). Wenn ja, gibt es `true` zurück.
2. Überprüft, ob das übergebene Objekt `null` ist oder ob die Klassen des aktuellen und des übergebenen Objekts unterschiedlich sind (`o == null || getClass() != o.getClass()`). Wenn eine dieser Bedingungen erfüllt ist, gibt es `false` zurück.
3. Castet das übergebene Objekt in ein `PersonBean`-Objekt.
4. Überprüft, ob die Attribute `vorname`, `nachname` und `geburtsdatum` des aktuellen `PersonBean`-Objekts gleich den entsprechenden Attributen des `PersonBean`-Objekts sind, das als Parameter übergeben wurde. Wenn alle Attribute gleich sind, gibt es `true` zurück, sonst `false`.

```
@Override  
public int hashCode()  
{  
    int result = vorname != null ? vorname.hashCode() : 0;  
    result = 31 * result + (nachname != null ? nachname.hashCode() : 0);  
    result = 31 * result + (geburtsdatum != null ?  
        geburtsdatum.hashCode() : 0);  
    return result;  
}
```

Abb.4 Methode hashCode

Die `hashCode()` Methode wurde zur `PersonBean` Klasse hinzugefügt. Diese Methode ist eine überschriebene Methode aus der `Object` Klasse in Java und wird verwendet, um einen eindeutigen Hashcode für das Objekt zu generieren.

In diesem Fall generiert die `hashCode()` Methode einen Hashcode basierend auf den Attributen `vorname`, `nachname` und `geburtsdatum` der `PersonBean`. Hier sind die Schritte, die es durchführt:

1. Es berechnet den Hashcode des `vorname`-Attributs, wenn es nicht null ist, sonst gibt es 0.
2. Es multipliziert das Ergebnis mit 31 (eine ungerade Primzahl) und addiert den Hashcode des `nachname`-Attributs, wenn es nicht null ist, sonst 0.
3. Es wiederholt den vorherigen Schritt für das `geburtsdatum`-Attribut.

1.4 Überwachung von Änderungen

```
private final PropertyChangeSupport propertyChangeSupport =  
new PropertyChangeSupport(this);  
private final VetoableChangeSupport vetoableChangeSupport =  
new VetoableChangeSupport(this);  
  
public void addPropertyChangeListener(PropertyChangeListener listener)  
{  
    propertyChangeSupport.addPropertyChangeListener(listener);  
}  
  
public void removePropertyChangeListener(PropertyChangeListener listener)  
{  
    propertyChangeSupport.removePropertyChangeListener(listener);  
}  
  
public void addVetoableChangeListener(VetoableChangeListener listener)  
{  
    vetoableChangeSupport.addVetoableChangeListener(listener);  
}  
  
public void removeVetoableChangeListener(VetoableChangeListener listener)  
{  
    vetoableChangeSupport.removeVetoableChangeListener(listener);  
}
```

Abb.5 Felder und Methoden zur Überwachung von Änderungen

PropertyChangeSupport: Dieses Feld wird verwendet, um Unterstützung für das Abfeuern von Eigenschaftsänderungsereignissen zu liefern. Es speichert eine Liste von `PropertyChangeListener`-Objekten, die benachrichtigt werden, wenn sich der Wert einer Eigenschaft ändert.

```
addPropertyChangeListener(PropertyChangeListener listener):
```

Diese Methode fügt einen PropertyChangeListener zur Liste hinzu.

```
removePropertyChangeListener(PropertyChangeListener listener):
```

Diese Methode entfernt einen PropertyChangeListener aus der Liste.

VetoableChangeSupport: Dieses Feld wird verwendet, um Unterstützung für das Abfeuern von "vetoable" Eigenschaftsänderungsereignissen zu liefern. Es speichert eine Liste von VetoableChangeListener-Objekten, die benachrichtigt werden, wenn sich der Wert einer Eigenschaft ändert, und die Möglichkeit haben, die Änderung abzulehnen.

```
addVetoableChangeListener(VetoableChangeListener listener):
```

Diese Methode fügt einen VetoableChangeListener zur Liste hinzu.

```
removeVetoableChangeListener(VetoableChangeListener listener):
```

Diese Methode entfernt einen VetoableChangeListener aus der Liste.

Diese Methoden ermöglichen es, dass andere Klassen auf Änderungen der Eigenschaften des PersonBean-Objekts reagieren können. Sie könnten beispielsweise verwendet werden, um eine Benutzeroberfläche zu aktualisieren, wenn sich der Wert einer Eigenschaft ändert, oder um die Änderung einer Eigenschaft unter bestimmten Bedingungen abzulehnen.

Anpassung der Überwachung auf die Setter von Vorname und Geburtsdatum:

```
public void setVorname(String vorname)
{
    String old = this.vorname;
    this.vorname = vorname;
    propertyChangeSupport.firePropertyChange("vorname", old, this.vorname);
}
```

Abb.6 angepasster Setter des Vornamens

Die setVorname(String vorname) Methode wurde aktualisiert, um Unterstützung für Eigenschaftsänderungsereignisse hinzuzufügen. Hier sind die Schritte, die es durchführt:

Es speichert den aktuellen Wert des vorname-Attributs in der lokalen Variable `old`.

Es aktualisiert den Wert des vorname-Attributs auf den neuen Wert, der als Parameter übergeben wurde.

Es ruft die firePropertyChange(String propertyName, Object old, Object new) Methode des propertyChangeSupport-Objekts auf und übergibt den Namen der Eigenschaft ("vorname"), den alten Wert und den neuen Wert.

Die firePropertyChange Methode benachrichtigt alle registrierten PropertyChangeListener-Objekte über die Änderung der vorname-Eigenschaft. Jeder Listener hat dann die Möglichkeit, auf diese Änderung zu reagieren, z.B. durch Aktualisierung einer Benutzeroberfläche oder Durchführung einer anderen Aktion.

```
public void setGeburtsdatum(LocalDate geburtsdatum) throws PropertyVetoException
{
    vetoableChangeSupport.fireVetoableChange("geburtsdatum",
        this.geburtsdatum, geburtsdatum);

    this.geburtsdatum = geburtsdatum;
}
```

Abb.7 angepasster Setter des Geburtsdatum

Die `setGeburtsdatum(LocalDate geburtsdatum)` Methode wurde aktualisiert, um Unterstützung für “vetoable” Eigenschaftsänderungsereignisse hinzuzufügen. Hier sind die Schritte, die es durchführt:

Es ruft die `fireVetoableChange(String propertyName, Object old, Object new)` Methode des `vetoableChangeSupport`-Objekts auf und übergibt den Namen der Eigenschaft ("geburtsdatum"), den alten Wert und den neuen Wert.

Es aktualisiert den Wert des `geburtsdatum`-Attributs auf den neuen Wert, der als Parameter übergeben wurde.

Die `fireVetoableChange` Methode benachrichtigt alle registrierten `VetoableChangeListener`-Objekte über die vorgeschlagene Änderung der `geburtsdatum`-Eigenschaft. Jeder Listener hat dann die Möglichkeit, die Änderung abzulehnen, indem er eine `PropertyVetoException` wirft. Wenn kein Listener die Änderung ablehnt, wird das `geburtsdatum`-Attribut auf den neuen Wert gesetzt.

1.5 Client Klasse

Anpassung Methode propertyChange

```
@Override
public void propertyChange(PropertyChangeEvent evt)
{
    System.out.println("Etwas hat sich geändert!");
    System.out.println(evt);
}
```

Abb.8 Methode propertyChange

Die `propertyChange(PropertyChangeEvent evt)` Methode ist eine Implementierung der `propertyChange` Methode aus dem `PropertyChangeListener` Interface. Diese Methode wird aufgerufen, wenn sich der Wert einer Eigenschaft des beobachteten Objekts ändert.

In diesem Fall druckt die Methode eine Nachricht “Etwas hat sich geändert!” und die Details des `PropertyChangeEvent` auf die Konsole. Ein `PropertyChangeEvent` ist ein Ereignisobjekt, das Informationen über die Änderung einer Eigenschaft enthält, einschließlich des Namens der Eigenschaft, des alten Werts und des neuen Werts.

Anpassung Methode vetoableChange

```
@Override
public void vetoableChange(PropertyChangeEvent evt) throws PropertyVetoException
{
    System.out.println("Etwas hat sich geändert!");
    System.out.println(evt);

    if (evt.getOldValue() != null)
        throw new PropertyVetoException("Alter Wert ist nicht Null!", evt);
}
```

Abb.9 Methode vetoableChange

Die `vetoableChange(PropertyChangeEvent evt)` Methode ist eine Implementierung der `vetoableChange` Methode aus dem `VetoableChangeListener` Interface. Diese Methode wird aufgerufen, wenn sich der Wert einer Eigenschaft des beobachteten Objekts ändern soll.

In diesem Fall druckt die Methode eine Nachricht "Etwas hat sich geändert!" und die Details des `PropertyChangeEvent` auf die Konsole. Ein `PropertyChangeEvent` ist ein Ereignisobjekt, das Informationen über die vorgeschlagene Änderung einer Eigenschaft enthält, einschließlich des Namens der Eigenschaft, des alten Werts und des neuen Werts.

Wenn der alte Wert der Eigenschaft nicht null ist, wirft die Methode eine `PropertyVetoException` mit der Nachricht "Alter Wert ist nicht Null!". Dies bedeutet, dass die Änderung der Eigenschaft abgelehnt wird, wenn der alte Wert nicht null ist.

Main Methode

In der `main` Methode der Client Klasse werden mehrere Konzepte und Funktionen von JavaBeans und der Java-Serialisierung demonstriert:

Erstellung von Objekten: Ein Client und ein `PersonBean` Objekt werden erstellt. Das `PersonBean` Objekt wird mit dem Vornamen "Vorname", dem Nachnamen "Ahuja" und dem Geburtsdatum 12. Mai 1990 initialisiert.

Hinzufügen von Listnern: Der Client wird als `PropertyChangeListener` und `VetoableChangeListener` zum `PersonBean` Objekt hinzugefügt. Dies bedeutet, dass der Client benachrichtigt wird, wenn sich eine Eigenschaft des `PersonBean` Objekts ändert oder eine Änderung vorgeschlagen wird.

Ändern von Eigenschaften: Der Vorname des `PersonBean` Objekts wird auf "Kamya" geändert. Dann wird versucht, das Geburtsdatum auf den 1. Januar 1700 zu ändern. Da der Client als `VetoableChangeListener` hinzugefügt wurde und die Änderung ablehnt, wenn der alte Wert nicht null ist, wird eine `PropertyVetoException` geworfen und die Änderung wird nicht durchgeführt.

Serialisierung: Das `PersonBean` Objekt wird in einen Byte-Stream umgewandelt und in einer Datei namens "person" gespeichert. Dies wird mit einem `FileOutputStream` und einem `ObjectOutputStream` durchgeführt.

Deserialisierung: Das PersonBean Objekt wird aus der Datei "person" gelesen und in ein Objekt umgewandelt. Dies wird mit einem FileInputStream und einem ObjectInputStream durchgeführt. Das resultierende Objekt wird dann in ein PersonBean Objekt gecastet, sofern es eine Instanz von PersonBean ist.

```
public static void main(String[] args)
{
    Client client = new Client();
    PersonBean person = new PersonBean("Vorname", "Ahuja",
    LocalDate.of(1990, 5, 12));
    person.addPropertyChangeListener(client);
    person.addVetoableChangeListener(client);
    System.out.println(person);

    person.setVorname("Kamya");

    try
    {
        person.setGeburtsdatum(LocalDate.of(1700,1,1));
    }
    catch (PropertyVetoException e)
    {
        System.out.println(e.getMessage());
    }
    System.out.println(person);

    System.out.println("\nSerialisierung:");

    try (FileOutputStream fileOutputStream =
    new FileOutputStream("person");
        ObjectOutputStream out = new ObjectOutputStream(fileOutputStream))
    {
        out.writeObject(person);
    }
    catch (IOException e)
    {
        e.printStackTrace();
    }

    person = null;

    try (FileInputStream fileInputStream = new FileInputStream("person");
        ObjectInputStream in = new ObjectInputStream(fileInputStream))
    {
        Object o = in.readObject();
        if (o instanceof PersonBean)
            person = (PersonBean) o;
    }
    catch (IOException | ClassNotFoundException e)
    {
        throw new RuntimeException(e);
    }
    System.out.println(person);
}
```

Abb.10 Main-Methode mit verschiedenen Konzepten von JavaBeans